REALTHREAD / TECHNICAL DOCUMENTATION

GD32H77D-Gino

开发文档

GD32H77D Gino 开发板使用指南与示例工程文档。

文档版本 **1.0.0**

更新时间 2026-10-21

2026, CYFS

目录

开发指南

1. GD32H77D Gino SDK 使用指南 . . . 1
   - 1.1 平台与开发环境 . . . 1
   - 1.2 硬件映射 . . . 2
   - 1.3 内存与下载布局 . . . 3
   - 1.4 RT-Thread Studio . . . 3
   - 1.5 Env 与 MDK . . . 4
   - 1.6 示例使用 . . . 4

快速开始

1. Gino_template . . . 7
   - 1.1 简介 . . . 7
   - 1.2 GPIO 与 RTOS 线程调度详解 . . . 7
   - 1.3 GD32H77D 的 GPIO 与 SysTick 特性 . . . 7
   - 1.4 RT-Thread PIN 与线程接口 . . . 8
   - 1.5 硬件说明 . . . 8
   - 1.6 工程示例说明 . . . 8
   - 1.7 运行效果 . . . 9

外设驱动

1. Gino_driver_can . . . 10
   - 1.1 简介 . . . 10
   - 1.2 CAN 总线、仲裁与错误处理详解 . . . 10
   - 1.3 GD32H77D 的 CAN1 特性 . . . 11
   - 1.4 RT-Thread CAN 设备接口 . . . 11
   - 1.5 硬件说明 . . . 11
   - 1.6 工程示例说明 . . . 11
   - 1.7 运行效果 . . . 14
2. Gino_driver_eth . . . 15
   - 2.1 简介 . . . 15
   - 2.2 Ethernet、RMII 与 TCP/IP 分层详解 . . . 15
   - 2.3 GD32H77D 的 ENET1 与 RMII 特性 . . . 16
   - 2.4 RT-Thread Ethernet、Netdev 与 SAL 设备接口 . . . 16
   - 2.5 硬件说明 . . . 16
   - 2.6 工程示例说明 . . . 16
   - 2.7 运行效果 . . . 17
3. Gino_driver_i2c . . . 18
   - 3.1 简介 . . . 18
   - 3.2 I2C 多主从总线协议详解 . . . 18
   - 3.3 GD32H77D 的 I2C1 特性 . . . 18
   - 3.4 RT-Thread I2C 总线设备接口 . . . 19
   - 3.5 硬件说明 . . . 19
   - 3.6 工程示例说明 . . . 20
   - 3.7 运行效果 . . . 21
4. Gino_driver_pwm . . . 22
   - 4.1 简介 . . . 22
   - 4.2 PWM 定时器输出原理详解 . . . 22

4.3 GD32H77D 的定时器 PWM 特性 . . . 22
4.4 RT-Thread PWM 设备接口 . . . 22
4.5 硬件说明 . . . 23
4.6 工程示例说明 . . . 23
4.7 运行效果 . . . 24
5. Gino_driver_qspi_flash . . . 25
5.1 简介 . . . 25
5.2 QSPI NOR Flash 协议与文件系统详解 . . . 25
5.3 GD32H77D 的 OSPI0 与 MDMA 特性 . . . 25
5.4 RT-Thread FAL、块设备与 DFS 设备接口 . . . 25
5.5 硬件说明 . . . 26
5.6 工程示例说明 . . . 26
5.7 运行效果 . . . 27
6. Gino_driver_rtc . . . 28
6.1 简介 . . . 28
6.2 RTC 实时时钟与 Alarm 详解 . . . 28
6.3 GD32H77D 的 RTC 与备份域特性 . . . 28
6.4 RT-Thread RTC 与 Alarm 设备接口 . . . 29
6.5 硬件说明 . . . 29
6.6 工程示例说明 . . . 29
6.7 运行效果 . . . 30
7. Gino_driver_sdcard . . . 31
7.1 简介 . . . 31
7.2 SD/SDIO 块设备协议详解 . . . 31
7.3 GD32H77D 的 SDIO1 特性 . . . 31
7.4 RT-Thread SDIO、块设备与 DFS 设备接口 . . . 31
7.5 硬件说明 . . . 32
7.6 工程示例说明 . . . 32
7.7 运行效果 . . . 33
8. Gino_driver_sdram . . . 34
8.1 简介 . . . 34
8.2 SDRAM 时序、刷新与内存扩展详解 . . . 34
8.3 GD32H77D 的 EXMC 与 SDRAM 特性 . . . 34
8.4 RT-Thread 内存管理接口 . . . 35
8.5 硬件说明 . . . 35
8.6 工程示例说明 . . . 35
8.7 运行效果 . . . 36
9. Gino_driver_spi . . . 37
9.1 简介 . . . 37
9.2 SPI 同步串行协议详解 . . . 37
9.3 GD32H77D 的 SPI3 特性 . . . 37
9.4 RT-Thread SPI 总线设备接口 . . . 38
9.5 硬件说明 . . . 38
9.6 工程示例说明 . . . 38
9.7 运行效果 . . . 39
10. Gino_driver_usb_device . . . 40
10.1 简介 . . . 40
10.2 USB Device 枚举与 CDC 类详解 . . . 40
10.3 GD32H77D 的 USBHS0 Device 特性 . . . 40
10.4 RT-Thread USB Device 设备接口 . . . 41
10.5 硬件说明 . . . 41
10.6 工程示例说明 . . . 41
10.7 运行效果 . . . 42

11. Gino_driver_usb_host . . . 43
11.1 简介 . . . 43
11.2 USB Host 枚举与大容量存储详解 . . . 43
11.3 GD32H77D 的 USBHS1 Host 特性 . . . 44
11.4 RT-Thread USB Host 与 DFS 设备接口 . . . 44
11.5 硬件说明 . . . 44
11.6 工程示例说明 . . . 44
11.7 运行效果 . . . 45

显示与摄像头

1. Gino_display_camera . . . 46
1.1 简介 . . . 46
1.2 采集与显示 . . . 46
1.3 内存要求 . . . 46
1.4 设备接口 . . . 46
1.5 硬件说明 . . . 46
1.6 示例使用 . . . 47
1.7 预期效果 . . . 47
2. Gino_display_gd32embeddedui . . . 48
2.1 可编辑的 UI 软件包 . . . 48
2.2 硬件与运行流程 . . . 48
2.3 打开板级工程 . . . 48
2.4 后续修改界面 . . . 49
2.5 上板检查 . . . 49
3. Gino_display_lvgl . . . 50
3.1 简介 . . . 50
3.2 LVGL 渲染、输入与双缓冲详解 . . . 50
3.3 GD32H77D 的 SDRAM、TLI/DSI 与 I2C3 特性 . . . 50
3.4 RT-Thread LVGL、Graphic 与 Touch 设备接口 . . . 51
3.5 硬件说明 . . . 51
3.6 工程示例说明 . . . 51
3.7 运行效果 . . . 52
4. Gino_display_lvgl8 . . . 53
4.1 简介 . . . 53
4.2 LVGL 渲染、输入与双缓冲详解 . . . 53
4.3 GD32H77D 的 SDRAM、TLI/DSI 与 I2C3 特性 . . . 53
4.4 RT-Thread LVGL、Graphic 与 Touch 设备接口 . . . 54
4.5 硬件说明 . . . 54
4.6 工程示例说明 . . . 54
4.7 运行效果 . . . 55

组件示例

1. Gino_component_mqtt . . . 56
1.1 简介 . . . 56
1.2 MQTT 发布订阅协议详解 . . . 58
1.3 GD32H77D 的 UART4 与 Cortex-M7 特性 . . . 58
1.4 RT-Thread AT Device、SAL 与 MQTT 设备接口 . . . 58
1.5 硬件说明 . . . 58
1.6 工程示例说明 . . . 58
1.7 运行效果 . . . 59

出厂演示

1. Gino_factory 60
   1.1 简介 60
   1.2 多外设并发与系统集成详解 64
   1.3 GD32H77D 的多总线、多 DMA 与存储器特性 64
   1.4 RT-Thread 综合设备与组件接口 64
   1.5 硬件说明 64
   1.6 工程示例说明 64
   1.7 运行效果 65

开发指南

1. GD32H77D Gino SDK 使用指南

English | SDK 概览

本文介绍 SDK 目录、工程选择、硬件连接、工程创建、编译下载和示例使用方法。

1.1 平台与开发环境

- 开发板：GD32H77D Gino；MCU：GD32H77DIW，Arm Cortex-M7，最高 600 MHz。
- RT-Thread：5.3.0。
- RT-Thread Studio 工具链：GNU Arm Embedded 13.3。
- Env：2.0 或更新版本，并配置 GNU Arm Embedded 工具链。
- MDK：Keil MDK 5.43 或兼容版本、Arm Compiler 6、GigaDevice GD32H77x_78x_DFP 0.6.1，以及 DAP-Link（CMSIS-DAP）或 J-Link。
- 下载接口：SWD；Studio 支持 DAP-Link/PyOCD 和 J-Link，MDK 支持 DAP-Link（CMSIS-DAP）和 J-Link。
- J-Link：当前使用版本为 v9.76a。
- 控制台：UART1，PA2/PA3，AF7，115200-8-N-1。
- 工程数量：17 个，包括 Gino_template 和 16 个示例。

SDK 目录

各示例共享 RT-Thread 内核、GD32 库、板级驱动和离线软件包。以下路径相对于 SDK 根目录。

目录	内容
projects/<name>/applications	应用入口与示例代码
projects/<name>/board	工程板级配置与链接脚本
libraries/Board_Drivers	共享板级初始化与板载外设驱动
libraries/gd32_drivers	GD32 外设的 RT-Thread 驱动
libraries/gd32-arm-*	GD32 CMSIS、启动代码与外设固件库
rt-thread	RT-Thread 内核、组件与构建工具
packages	示例使用的离线软件包

每个工程拥有独立的 .config、rtconfig.h、Studio 元数据和 MDK 工程文件。功能配置在所选工程中修改；共享驱动的修改会影响使用该驱动的其他工程。

首次编译或新建应用建议从 Gino_template 开始。验证外设时，按第 6 节选择对应的独立示例；体验开发板综合功能时，使用 Gino_factory。

1.2 硬件映射

外设	引脚或资源	说明
运行 LED	PC4	每 500 ms 翻转
UART1	PA2/PA3，AF7	控制台
UART4	PB12/PB13，AF14	GD32VW553 AT 模块
I2C1	PH4/PB11，AF4	独立 I2C 示例
I2C2	PA8/PC9，AF4	OV7670 控制
I2C3	PD12/PD13，AF4	GT911 触摸
SPI3	PE12/PF0/PF1	SCK/MISO/MOSI
CAN1	PB4/PB5	需要外接 CAN 收发器和终端电阻
LCD 背光	TIMER1 CH0，PA0	pwm1 通道 1
PWM 输出	PA7 与 PG7	pwm2 通道 2 和 pwm30 通道 3
GT911	reset PH3，interrupt PH5	地址0x5D
OV7670	XCLK PE9，I2C2，DCI + DMA1 通道 7	QVGA RGB565
QSPI Flash	CS PC11，CLK PF10，D0-D3 PF8/PF9/PF7/PA1	8 MiB GD25Q64E，连接 OSPI0
USB Device	USBHS0，PA11/PA12	Type-C
USB Host	USBHS1，PB14/PB15，VBUS PF2	USB-A
以太网	ENET1 RMII	PHY 地址 2，PC12 输入 50 MHz REF_CLK
SDRAM	0xC0000000，32 MiB	显示缓冲和扩展 heap

外接模块的供电、共地和接口电平要求，以对应工程 README 的硬件说明为准。

1.3 内存与下载布局

区域	地址	大小	用途
CNVM	0x08010000	1984 KiB	应用向量表、启动代码和主程序
ECNVM	0x08200000	7680 KiB	扩展程序 Flash，由链接脚本分配
ITCMRAM	0x00000000	128 KiB	按链接脚本分配
DTCMRAM	0x20000000	256 KiB	按链接脚本分配
AXISRAM	0x24000000	768 KiB	data、BSS、栈、内部 heap 和摄像头缓冲
SRAM1	0x30004000	16 KiB	以太网 DMA 描述符和缓冲
SDRAM	0xC0000000	32 MiB	显示缓冲和扩展 heap

应用向量表位于 0x08010000，0x08000000 的启动入口用于兼容不同芯片版本。默认生成的 BIN 镜像包含该启动入口，应用向量表位于镜像偏移 0x10000。将完整 BIN 镜像下载到 0x08000000。

启用 BSP_USING_LCD_MIPI 时，SDRAM 前 3 MiB 为预留区，扩展 heap 从 0xC0300000 开始；未启用 LCD 时，SDRAM heap 从 0xC0000000 开始。摄像头采集和预览缓冲使用内部 AXI SRAM，与 SDRAM 显示区域分开管理。

1.4 RT-Thread Studio

1. 安装 sdk-bsp-gd32h77d-gino.yaml 描述的 GD32H77D-Gino BSP **v1.0.0**。
2. 新建基于 GD32H77D-Gino 开发板的 RT-Thread 工程。
3. 选择 Gino_template 或 16 个示例中的一个。
4. 在 RT-Thread Settings 中配置需要的外设和软件包，使用 GNU Arm Embedded 13.3 构建。
5. 连接 UART1，设置为 115200-8-N-1，并连接 SWD 下载器。
6. 选择 DAP-Link、目标 GD32H77DIW 和 BIN 模式，将下载起始地址设为 0x08000000，下载 Debug/rtthread.bin。SDK 默认使用这些下载设置，构建时保留 Debug/rtthread.elf 用于调试。
7. 复位开发板，按下文运行对应示例。

Studio 创建出的工程包含该示例需要的共享源码目录。

1.5 Env 与 MDK

Env 与 GCC

在 Env PowerShell 环境中，从 SDK 根目录进入目标工程。以模板为例：

```
cd projects\Gino_template
.\mklinks.bat
$env:RTT_EXEC_PATH = "<gnu-arm-toolchain>\bin"
scons -j8
```

将工具链路径替换为包含 arm-none-eabi-gcc 的目录。mklinks.bat 为共享的 rt-thread 和 libraries 创建目录链接。SCons 构建生成 rt-thread.elf、rtthread.hex 和 rtthread.bin，将 rtthread.bin 下载到 0x08000000。

MDK5 与 Arm Compiler 6

安装第 1 节列出的 MDK 依赖，在 Env PowerShell 环境中从 SDK 根目录进入目标工程：

```
cd projects\Gino_template
.\mklinks.bat
scons --target=mdk5
```

template.uvprojx 和 template.uvoptx 保存 uVision 默认配置。SCons 根据当前配置生成源码分组、包含路径和宏定义。配置或源码选择变更后需要重新生成 uVision 工程，生成的源码分组会被覆盖。

1.6 示例使用

各示例的接线、配置、命令和预期现象见侧栏中的对应工程文档。下表汇总运行前置条件和首次操作。

工程	前置条件	操作或预期行为
Gino_template	开发板供电	PC4 闪烁，UART1 控制台可用
Gino_driver_i2c	上拉电阻和 I2C 目标设备	i2c scan hwi2c1 08 78
Gino_driver_spi	SPI 目标设备及应用配置的 CS	使用spi3 总线，在应用中配置从设备协议
Gino_driver_qspi_flash	板载 GD25Q64E	ls /flash 访问文件
Gino_driver_sdcard	FAT/FAT32 SD 卡	ls /sd
Gino_driver_can	CAN 收发器、终端电阻和通信对端	使用can1，波特率和帧格式与对端一致
Gino_driver_pwm	按需连接输出端	pwm_dual_test start，PA7/PG7 输出 PWM
Gino_driver_rtc	RTC 时钟源	date
Gino_driver_sdram	板载 SDRAM	外部内存加入 heap；free、list_memheap
Gino_driver_usb_device	支持数据传输的 USB 线	主机枚举配置的 USB class
Gino_driver_usb_host	FAT U 盘	ls /udisk
Gino_driver_eth	网线及可用网络	ifconfig、ping <gateway>
Gino_display_lvgl	LCD、触摸和 SDRAM	自动启动 LVGL 9.x demo，通过触摸操作
Gino_display_lvgl8	LCD、触摸和 SDRAM	自动启动 LVGL 8.3.11 demo，通过触摸操作
Gino_display_camera	OV7670、LCD 和 SDRAM	自动预览；ov7670_preview start / ov7670_preview stop
Gino_component_mqtt	已配置 Wi-Fi 和 broker	gd32vw553_mqtt_start、gd32vw553_mqtt_pub <topic> <message>
Gino_factory	已启用的板级外设	使用综合界面及摄像头、存储和网络示例

Flash 文件系统

filesystem FAL 分区挂载到 /flash。对于需要格式化的新分区：

```
mkfs -t elm filesystem
mount filesystem /flash elm
ls /flash
```

格式化会清除分区原有数据。FatFs 需要允许最大 4096 字节扇区。

Wi-Fi 与 MQTT

在 RT-Thread Settings 或 menuconfig 中配置 Wi-Fi SSID/password、broker 地址/端口、topic、client ID 和可选认证信息。真实凭据保存在本地配置中。当模块固件不支持 AT+CIPDOMAIN DNS 查询时，使用数字 IPv4 broker 地址。

显示与摄像头

Gino_display_lvgl（9.x）和 Gino_display_lvgl8（8.3.11）自动启动 LVGL 音乐 demo。Gino_display_camera 在 LVGL 初始化后请求预览，显示 QVGA RGB565 图像。使用 ov7670_preview stop 和 ov7670_preview start 控制预览。LCD 帧缓冲位于 SDRAM 预留区，应用动态内存应从配置的 heap 分配。

快速开始

1. Gino_template

1.1 简介

本工程是 GD32H77D Gino 开发板的 LED 闪烁与二次开发模板，用于单独学习、配置和验证 GPIO 与 RTOS 线程调度。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

本工程没有独立的主设备节点。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的运行探测状态。	本模板没有独立的主参考设备节点。
pin list	查看 PIN 设备信息。	确认 GPIO/PIN 注册。

1.2 GPIO 与 RTOS 线程调度详解

启动汇编完成数据段初始化后，RT-Thread 配置时钟、控制台、系统 tick 和调度器，再进入应用 main。应用通过 PIN 设备接口驱动 PC4，并用 rt_thread_mdelay 主动让出 CPU；所以 LED 闪烁同时验证了 GPIO、系统节拍和线程调度。

一次完整的数据路径如下：

```
复位 -> board 初始化 -> RT-Thread 调度器 -> main 线程 -> rt_pin_write -> PC4 LED
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

1.3 GD32H77D 的 GPIO 与 SysTick 特性

GD32H77D 的 GPIO 支持输入、推挽/开漏输出和复用功能。板载运行指示灯连接 PC4；BSP 在 board.c 中完成系统时钟、SysTick、PIN 驱动和 UART1 控制台初始化，因此 PC4 翻转也能间接验证系统节拍与主线程调度。

1.4 RT-Thread PIN 与线程接口

RT-Thread PIN 接口使用 rt_pin_mode 配置方向，使用 rt_pin_write/rt_pin_read 操作电平。示例在线程中调用 rt_thread_mdelay(500) 让出处理器，而不是用空循环阻塞 CPU。

本工程没有独立的主参考设备节点。可先通过 list_device 和 gino_device_probe 判断运行状态；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

1.5 硬件说明

无需额外硬件，使用 PC4 LED 验证系统节拍、GPIO 和基础启动。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

1.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/main.c
- applications/device_probe.c
- ../../libraries/Board_Drivers/board.c
- board/SConscript

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_template
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- pin list

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

1.7 运行效果

预期现象

- UART1 输出 RT-Thread 启动信息和工程名称。
- PC4 LED 每 500 ms 翻转一次，完整亮灭周期约 1 s。
- pin list 能显示 PIN 设备和当前引脚信息。

外设驱动

1. Gino_driver_can

1.1 简介

本工程是 GD32H77D Gino 开发板的 CAN1 外部总线收发示例。上电只初始化设备并启动接收，不自动发送。每次执行 gino_can_send 命令发送一帧经典 CAN；启用 CAN FD 时，再发送一帧 CAN FD。命令可以反复调用，外部节点发来的数据通过彩色 ulog 打印到串口终端。

主参考设备：can1。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_can_send	发送一帧经典 CAN，启用 CAN FD 时再发送一帧 CAN FD。	每次调用只发送一组，可反复执行。
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 can1。
list_device	查看系统已注册设备。	确认 UART、PIN、总线和目标设备存在。

1.2 CAN 总线、仲裁与错误处理详解

CAN1 控制器负责位时序、ID 仲裁、ACK、错误计数和重发，PB4/PB5 仍需通过外部 CAN 收发器连接差分 CANH/CANL。低数值 ID 在仲裁中通常具有更高优先级，应用需要自行定义 ID 和负载协议。

一次完整的数据路径如下：

```
应用 CAN 帧 -> RT-Thread can1 -> CAN1 控制器 -> PB4/PB5 -> 收发器 -> CANH/CANL
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

1.3 GD32H77D 的 CAN1 特性

工程启用 GD32H77D CAN1，PB4/PB5 连接控制器 TX/RX。控制器提供位时序、过滤器、发送邮箱、接收 FIFO、错误计数和自动重发，但物理差分层必须由外部 CAN 收发器完成。

1.4 RT-Thread CAN 设备接口

RT-Thread 将 CAN1 注册为 can1。应用通过 rt_device_open 配置中断收发模式，通过 rt_device_control 设置波特率/模式，使用 struct rt_can_msg 配合 rt_device_read/rt_device_write 收发帧。

主参考设备：can1。应用在正常模式下打开设备，MSH 命令按顺序发送帧，接收回调通知接收线程读取队列并打印数据。

1.5 硬件说明

CAN1 默认使用 PB4/AF4 发送、PB5/AF9 接收。通过外部收发器连接对端的 CANH/CANL，并连接参考地，总线两端各配置 120 Ω 终端电阻。启用 CAN FD 时，收发器和对端都必须支持 ISO CAN FD 及 2 Mbit/s 数据段速率。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

必须外接 CAN 收发器并正确配置终端电阻；MCU 引脚不能直接连接 CANH/CANL。

1.6 工程示例说明

上电只启动接收。需要发送时，在 MSH 终端执行：

```
msh /> gino_can_send
```

每次调用只发送一组，执行结束后返回命令行；再次输入同一命令即可再次发送。接收线程始终运行。

- applications/main.c：初始化 CAN、顺序发送经典 CAN/CAN FD 帧、接收并打印外部数据。
- applications/device_probe.c
- ../../libraries/gd32_drivers/drv_can.c
- ../../libraries/gd32_drivers/config/can_config.h

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_can
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

命令发送的数据

默认仲裁段为 **500 kbit/s**。定义 RT_CAN_USING_CANFD 时，应用自动启用 ISO CAN FD，数据段为 **2 Mbit/s**，并保留经典 CAN 帧的收发能力。

发送顺序	帧类型	标准 ID	长度	BRS	数据
1	经典 CAN 数据帧	0x123	8 字节	0	00 01 02 03 04 05 06 07
2	CAN FD 数据帧，仅启用 CAN FD 时发送	0x124	64 字节	1	00 至 3F，每字节递增 1

每次执行 gino_can_send 先发送一帧经典 CAN，启用 CAN FD 时等待 100 ms 后再发送一帧 CAN FD，随后结束，不循环发送。关闭 RT_CAN_USING_CANFD 后，每次命令只发送第一帧。发送成功通过 LOG_I 打印绿色 [TX]，失败或等待完成超时通过 LOG_E 打印红色 [TX FAIL]，应用不会自动重试该命令。

接收并打印到终端

接收线程持续读取外部节点发送的帧，每收到一帧就通过 LOG_I 输出一行绿色 [RX]，包含帧类型、标准/扩展 ID、RTR、BRS、字节长度及全部十六进制数据。打印操作在接收线程中执行，中断回调只负责通知。

工程已启用 RT_USING_ULOG、ULOG_BACKEND_USING_CONSOLE 和 ULOG_USING_COLOR。日志包含 tick 时间、等级和 can.demo 标签；ULOG_LINE_BUF_SIZE=512 可完整容纳 64 字节 FD 数据。串口终端需支持 ANSI 颜色显示。

这是正常模式下的外部收发，开发板不会将自己发送的帧作为接收数据打印。要同时观察对应的 [TX] 和 [RX]，请将对端配置为收到数据后原样回发，或在 CAN 工具中手动发送同样的 ID 和数据。仅由对端提供 ACK 不会产生应用层接收帧。开发板只打印收到的帧，不自动回发接收数据。

执行命令且对端原样回发时，可观察到如下日志正文（省略 ulog 时间、等级和标签前缀）。实际 TX/RX 日志的先后顺序取决于对端响应时机：

```
can1 normal mode: arbitration=500000 bit/s, ISO CAN FD data=2000000 bit/s
[TX] can1 CAN STD ID=0x00000123 RTR=0 BRS=0 LEN=8 DATA=00 01 02 03 04 05 06 07
[RX] can1 CAN STD ID=0x00000123 RTR=0 BRS=0 LEN=8 DATA=00 01 02 03 04 05 06 07
[TX] can1 CANFD STD ID=0x00000124 RTR=0 BRS=1 LEN=64 DATA=00 01 02 03 04 05 06 07
  08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21
  22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B
```

(续下页)

(接上页)

```
  3C 3D 3E 3F
[RX] can1 CANFD STD ID=0x00000124 RTR=0 BRS=1 LEN=64 DATA=00 01 02 03 04 05 06 07
  08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20 21
  22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B
  3C 3D 3E 3F
```

CAN FD 配置说明

本工程已启用 RT_CAN_USING_CANFD，非阻塞发送缓冲区为 1152 字节，可存放 16 个 rt_can_msg；本示例使用阻塞发送保证发送顺序。其他工程可在 RT-Thread Components -> Device Drivers -> Using CAN device drivers 中启用 Enable CAN-FD support。

can_demo_init() 根据编译选项配置 enable_canfd，再打开正常模式下的中断收发。发送经典 CAN 时 fd_frame=0，随后发送 FD 帧时设置 fd_frame=1、brs=1，两种帧之间不需要重新初始化控制器。

- message.len 表示字节数，64 字节填写 64，不是 DLC 编码 15。合法长度为 0 至 8、12、16、20、24、32、48、64；其他 9 至 63 字节长度会补零至下一个合法长度，接收端返回补齐后的长度。
- fd_frame=0 发送经典 CAN 帧，最多 8 字节；fd_frame=1, brs=0 发送不切换波特率的 FD 帧；fd_frame=1, brs=1 启用数据段速率切换。CAN FD 不支持远程帧。
- 接收仍使用 rt_device_read，返回的 fd_frame、brs 和 len 描述实际帧。启用硬件过滤宏时，读取普通接收队列前设置 hdr_index=-1。
- 数据段最高 8 Mbit/s，驱动根据实际 APB2 时钟选择仲裁段和数据段共用的预分频，约 80% 采样点。默认 300 MHz APB2 下支持 500 kbit/s 仲裁配合 2、4、5 Mbit/s 数据段；8 Mbit/s 无法由该时钟整除，会返回 -RT_EINVAL。自定义位时序接口暂不支持，返回 -RT_ENOSYS。
- CAN FD 的 64 字节邮箱占用 72 字节，512 字节消息 RAM 最多容纳 7 个邮箱。启用编译选项后固定使用邮箱 0 至 2 接收、3 至 6 发送，RT_CANSND_BOX_NUM 允许 1 至 3，至少保留一个邮箱用于非阻塞发送。关闭编译选项后保留经典 CAN 的 16 接收、16 发送布局。
- 用 RT_CAN_CMD_SET_CANFD 和 (void *)0U 切回经典 CAN。修改配置前停止发送并等待发送队列排空，硬件发送尚未完成时返回 -RT_EBUSY。

验证步骤

1. 连接收发器、CANH/CANL、参考地和终端电阻，打开 UART1 串口终端，设置为 115200-8-N-1。
2. 将外部 CAN 工具或另一块开发板设置为正常模式、仲裁段 500 kbit/s；本工程启用 CAN FD 时，对端同时设置 ISO CAN FD、数据段 2 Mbit/s。
3. 复位开发板，此时不发送。执行 gino_can_send，观察终端依次打印经典 CAN 和 CAN FD 的发送数据，使用对端确认收到 0x123 和 0x124；再次执行命令可再次发送一组。

4. 从对端发送对应帧或开启对端回发功能，确认开发板打印完整的 [RX] 数据。接收也支持其他 ID，无需限定为示例发送 ID。
5. 可执行 list_device 和 gino_device_probe 检查设备状态。若出现 [TX FAIL]，检查对端是否在线并提供 ACK、速率是否一致以及接线和终端电阻。

1.7 运行效果

预期现象

- list_device 和 gino_device_probe 能找到 can1。
- 上电不发送。每次执行 gino_can_send 先发送一帧 0x123 经典 CAN，启用 CAN FD 时再发送一帧 0x124 CAN FD；命令可以反复调用。
- 对端发送或回发数据后，终端逐帧打印 [RX] 和完整数据。
- 总线空闲时 CANH/CANL 处于正确的隐性电平。

2. Gino_driver_eth

2.1 简介

本工程是 GD32H77D Gino 开发板的 ENET1 RMII 以太网参考工程，用于单独学习、配置和验证 Ethernet、RMII 与 TCP/IP 分层。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：e0。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 e0。
ifconfig	查看网络设备的联网状态和 IP 地址。	确认网络已 ready。
ping 192.168.1.1	通过当前网络接口发送 ICMP 报文。	按实际网关或对端地址替换。

2.2 Ethernet、RMII 与 TCP/IP 分层详解

lwIP 的 pbuf 通过 RT-Thread eth_device 交给 ENET1。发送端复制 pbuf 链并把 DMA 描述符 ownership 交给硬件；接收中断只通知网络线程，线程复制有效帧并把描述符归还 DMA。PHY 线程负责地址检测、自动协商和链路状态。

一次完整的数据路径如下：

```
socket/SAL -> lwIP -> e0 eth_device -> ENET DMA -> RMII -> PHY -> 网线
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

2.3 GD32H77D 的 ENET1 与 RMII 特性

工程使用 GD32H77D ENET1 RMII，PHY 默认地址 2，PC12 接收 50 MHz REF_CLK。ENET DMA 使用专用描述符和缓冲区完成帧收发，MDIO/MDC 用于 PHY 寄存器管理和自动协商。

2.4 RT-Thread Ethernet、Netdev 与 SAL 设备接口

驱动注册 RT-Thread eth_device 和 e0 netdev。接收中断调用 eth_device_ready 唤醒网络线程，lwIP 提供 IP/TCP/UDP，SAL 向应用提供统一 socket，ifconfig 查看网络配置，ping 检查网络连通性。

主参考设备：e0。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

2.5 硬件说明

ENET1 使用 RMII，默认 PHY 地址 2；PHY 必须向 PC12 提供 50 MHz 参考时钟。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

2.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- ../../libraries/Board_Drivers/drv_enet.c
- board/SConscript
- rtconfig.h
- applications/device_probe.c

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_eth
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- ifconfig
- ping 192.168.1.1

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

2.7 运行效果

预期现象

- 启动日志能找到默认地址 2 的 PHY 并完成自动协商。
- ifconfig 显示 e0 link up 和有效 IP。
- ping < 网关> 有稳定回复，持续收发时无 DMA descriptor 错误。

3. Gino_driver_i2c

3.1 简介

本工程是 GD32H77D Gino 开发板的 I2C1 硬件总线参考工程，用于单独学习、配置和验证 I2C 多主从总线协议。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：hwi2c1。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查hwi2c1。
i2c scan hwi2c1	扫描 I2C1 总线并打印有响应的从设备地址。	由i2c-tools 软件包提供，连接已供电的 I2C 目标后使用。

工程已启用 i2c-tools v1.0.0（PKG_USING_I2C_TOOLS），使用硬件总线模式。打开工程前，Windows 执行 mklinks.bat，Linux 执行 sh mklinks.sh，建立仓库内软件包的目录链接。执行 i2c 可查看扫描、读写命令用法。

3.2 I2C 多主从总线协议详解

I2C 使用开漏 SDA/SCL 和上拉电阻形成可共享总线。控制器产生 START、地址、ACK、数据和 STOP；i2c scan 使用零数据长度的写传输，通过地址阶段是否收到 ACK 判断器件存在，不验证器件寄存器功能。

一次完整的数据路径如下：

```
MSH i2c scan -> i2c-tools -> hwi2c1 -> I2C1 控制器 -> PH4/PB11 -> 从设备 ACK
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

3.3 GD32H77D 的 I2C1 特性

工程使用 GD32H77D 硬件 I2C1，PH4/PB11 复用为 AF4。控制器负责 START/STOP、地址、ACK/NACK 和字节时序；SDA/SCL 仍必须依靠外部或板载上拉形成高电平。

3.4 RT-Thread I2C 总线设备接口

RT-Thread 把控制器注册为 hwi2c1 总线。器件驱动可调用 rt_i2c_transfer，或使用 rt_i2c_master_send/rt_i2c_master_recv；i2c scan hwi2c1 通过地址 ACK 扫描器件。

主参考设备：hwi2c1。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

3.5 硬件说明

I2C1 默认使用 PH4/PB11（AF4）。外部总线需要合适的上拉电阻。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

外部总线需要合适的上拉电阻，并与目标设备共地。



3.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/main.c
- applications/device_probe.c
- ../../libraries/gd32_drivers/drv_hard_i2c.c
- ../../libraries/gd32_drivers/config/i2c_config.h
- ../../packages/i2c-tools-v1.0.0/src/i2c_tools.c
- ../../packages/i2c-tools-v1.0.0/src/i2c_utils.c

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_i2c
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。本工程还会链接离线软件包 i2c-tools-v1.0.0。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- i2c scan hwi2c1 08 78

扫描范围按十六进制解析，结束地址不包含在内：08 78 扫描 0x08-0x77，跳过保留地址。不指定范围时，软件包扫描 0x00-0x7F。gino_device_probe 只检查总线注册，不探测从设备。

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

3.7 运行效果

预期现象

- list_device 中出现 hwi2c1。
- 接入已知地址器件后，i2c scan hwi2c1 08 78 能列出响应地址。
- 逻辑分析仪可看到 START、7 位地址、ACK 和 STOP。

4. Gino_driver_pwm

4.1 简介

本工程是 GD32H77D Gino 开发板的 PWM2 与 PWM30 双通道输出参考工程，用于单独学习、配置和验证 PWM 定时器输出原理。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：pwm2。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查pwm2。
pwm_dual_test start	启动双路 PWM 输出测试。	在 PA7 和 PG7 输出 PWM。
pwm_dual_test stop	停止双路 PWM 输出测试。	关闭测试输出。

4.2 PWM 定时器输出原理详解

PWM 由定时器周期寄存器决定频率，由比较寄存器决定高电平持续时间。示例同时驱动 pwm2 通道 2 和 pwm30 通道 3，并周期调整 pulse，用于验证两个定时器通道和引脚复用。

一次完整的数据路径如下：

```
pwm_dual_test -> RT-Thread PWM -> TIMER/PWM 比较输出 -> PA7 与 PG7
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

4.3 GD32H77D 的定时器 PWM 特性

GD32H77D 定时器使用计数周期决定 PWM 频率，比较值决定有效电平时间。工程提供 pwm2 通道 2（PA7）和 pwm30 通道 3（PG7），用于同时验证两路定时器输出。

4.4 RT-Thread PWM 设备接口

RT-Thread PWM 接口使用 rt_device_pwm_set/rt_pwm_set 设置 period 和 pulse，使用 rt_device_pwm_enable/rt_pwm_enable 启动通道，停止时调用对应 disable 接口。参数时间单位以当前 RT-Thread PWM API 定义为准。

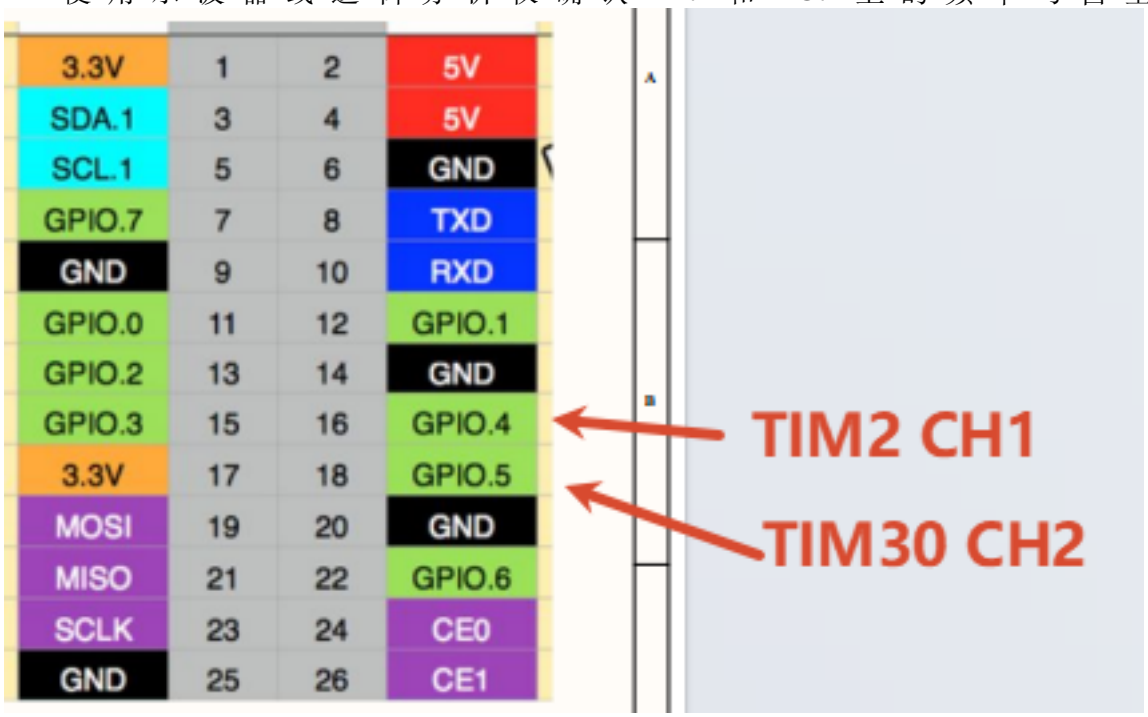
主参考设备：pwm2。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

4.5 硬件说明

pwm2 通道 2 输出到 PA7，pwm30 通道 3 输出到 PG7，可使用示波器或逻辑分析仪观察。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

使用示波器或逻辑分析仪确认 PA7 和 PG7 上的频率与占空比。



4.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/pwm_dual_test.c
- ../../libraries/gd32_drivers/drv_pwm.c
- board/Kconfig

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_pwm
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- pwm_dual_test start
- pwm_dual_test stop

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

4.7 运行效果

预期现象

- list_device 中出现 pwm2 和 pwm30。
- 启动测试后 PA7、PG7 输出约 2 kHz PWM。
- 示波器可观察到占空比按示例规律变化，停止后输出关闭。

5. Gino_driver_qspi_flash

5.1 简介

本工程是 GD32H77D Gino 开发板的 QSPI Flash 参考工程，用于验证通过 OSPI0 连接的板载 GD25Q64E。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：qspi_flash0。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 qspi_flash0。
fal probe filesystem	探测 FAL filesystem 分区。	访问文件系统前确认分区信息。
fal read 0 16	从当前 FAL 设备读取数据。	用于基础 flash 读取检查。

5.2 QSPI NOR Flash 协议与文件系统详解

OSPI0 以四线数据阶段访问 GD25Q64E NOR Flash。底层驱动提供读、页编程和扇区擦除，FAL 再把 Flash 划分为 filesystem 分区并转换成块设备，DFS/FatFs 最终挂载为 /flash。NOR Flash 写入前必须满足擦除和忙状态约束。

一次完整的数据路径如下：

```
GD25Q64E -> OSPI0/MDMA -> qspi_flash0 -> FAL filesystem -> 块设备 -> FatFs -> /
  flash
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

5.3 GD32H77D 的 OSPI0 与 MDMA 特性

GD32H77D OSPI0 可在命令、地址和数据阶段使用多线传输，并由 MDMA 减少大块搬运的 CPU 开销。板载 GD25Q64E 容量为 8 MiB，典型擦除粒度 4 KiB、页编程粒度 256 B。

5.4 RT-Thread FAL、块设备与 DFS 设备接口

底层注册 qspi_flash0，FAL 通过 fal_init 和 fal_partition_find 找到 filesystem 分区，再由 fal_blk_device_create 暴露块设备，DFS/FatFs 将其挂载到 /flash。

主参考设备：qspi_flash0。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

5.5 硬件说明

板载 8 MiB GD25Q64E 注册为 qspi_flash0，FAL 分区 filesystem 挂载到 /flash。首次使用可能需要格式化。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

未格式化分区可能需要先格式化才能挂载 /flash；格式化会清除已有内容。

5.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- ../../libraries/Board_Drivers/drv_qspi_flash.c
- ../../libraries/Board_Drivers/drv_filesystem.c
- ../../libraries/Board_Drivers/fal_cfg.h
- ../../libraries/gd32_drivers/drv_ospi.c

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_qspi_flash
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- fal probe filesystem
- fal read 0 16

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

5.7 运行效果

预期现象

- gino_device_probe 能找到 qspi_flash0。
- fal probe filesystem 显示分区地址和长度。
- fal read 0 16 能稳定读取分区开头；格式化并挂载后可访问 /flash。

6. Gino_driver_rtc

6.1 简介

本工程是 GD32H77D Gino 开发板的片上 RTC 与 Alarm 参考工程，用于单独学习、配置和验证 RTC 实时时钟与 Alarm。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：rtc。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 rtc。
date	通过 MSH date 命令读取或设置 RTC 时间。	重复执行可确认秒计数递增。
rtc_alarm [秒数]	设置一次性 RTC 闹钟。	默认 5 秒，范围 1..86400 秒；再次设置会替换本示例的旧闹钟。
rtc_alarm status	查看本示例闹钟状态和目标时间。	时间按本地时区显示。
rtc_alarm stop	取消并删除本示例闹钟。	不删除其他应用创建的闹钟。

6.2 RTC 实时时钟与 Alarm 详解

RTC 使用低速时钟和备份域独立计时，RT-Thread 将其注册为 rtc，date 通过统一 RTC 接口读取或设置时间。Alarm 由 RTC 比较逻辑产生唤醒/中断事件，不依赖主循环持续轮询。

一次完整的数据路径如下：

```
低速时钟/备份域 -> 片上 RTC -> rt_device rtc -> date/Alarm
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

6.3 GD32H77D 的 RTC 与备份域特性

GD32H77D 片上 RTC 位于低速时钟和备份域，可在主 CPU 停止时继续计时。Alarm 比较单元可以产生中断/唤醒；复位或掉电后的保持能力取决于备份域和 VBAT 条件。

6.4 RT-Thread RTC 与 Alarm 设备接口

RT-Thread 注册 rtc，时间读写通过 RTC device control 接口完成，POSIX 时间函数和 MSH date 命令建立在该设备之上。Alarm 组件管理闹钟对象并由 RTC 中断触发回调。

主参考设备：rtc。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

6.5 硬件说明

验证片上 RTC 与 Alarm；掉电保持能力取决于 RTC 时钟源、备份域和后备电源。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

RTC 保持能力取决于低速时钟、备份域和 VBAT 条件。

6.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/device_probe.c
- applications/rtc_alarm_example.c
- ../../libraries/gd32_drivers/drv_rtc.c
- board/Kconfig

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_rtc
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- date
- rtc_alarm 5
- rtc_alarm status

- rtc_alarm stop

```
date 2026 09 08 12 00 00
rtc_alarm 5
rtc_alarm status
```

约 5 秒后，ulog 输出一次带本地日期时间的 [I/rtc.alarm] Triggered: 日志。不带参数的 rtc_alarm 同样设置 5 秒后的闹钟，rtc_alarm help 查看用法。测试取消功能时，先执行 rtc_alarm 30，再在到期前执行 rtc_alarm stop。

命令通过 rt_alarm_create() 和 rt_alarm_start() 创建、启动一次性闹钟。RTC Alarm0 中断通知 RT-Thread Alarm 服务，由服务线程执行回调并打印日志。工程默认启用 INFO 级别的 ulog 控制台后端，Alarm 服务线程栈调整为 2048 字节。应用启动和设备探测日志也使用 ulog；系统自带的 date、list_device 保持标准命令行输出。

闹钟截止时间基于 RTC 时间计算，设置闹钟后应避免再用 date 调整时间。示例闹钟对象保存在 RAM 中，复位后需要重新设置，即使 RTC 时间仍然保持。

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

6.7 运行效果

预期现象

- gino_device_probe 能打开 rtc。
- 连续执行 date 时秒值单调增加。
- rtc_alarm 5 到期后仅输出一次 ulog 触发日志，取消后的闹钟不再触发。
- 设置时间后复位，是否保持应与备份域和后备电源设计一致。

7. Gino_driver_sdcard

7.1 简介

本工程是 GD32H77D Gino 开发板的 SDIO1 与 SD 卡文件系统参考工程，用于单独学习、配置和验证 SD/SDIO 块设备协议。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：sd0。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 sd0。
ls /sd	列出已挂载 SD 卡文件系统中的文件。	需要可用的 FAT/FAT32 卡。

7.2 SD/SDIO 块设备协议详解

SDIO1 完成 SD 卡识别和块读写后注册 sd0，DFS/FatFs 读取分区与文件系统元数据并挂载到 /sd。卡检测脚只说明介质是否插入，不能代替初始化、块读写和文件系统验证。

一次完整的数据路径如下：

```
SD 卡 -> SDIO1 + 卡检测 -> sd0 块设备 -> DFS/FatFs -> /sd
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

7.3 GD32H77D 的 SDIO1 特性

工程启用 GD32H77D SDIO1 及卡检测，控制器完成命令响应、数据线传输和块读写。卡检测配置为低电平有效，但卡检测只表示介质插入，不代表卡初始化和文件系统已成功。

7.4 RT-Thread SDIO、块设备与 DFS 设备接口

RT-Thread SDIO/MMCSd 层在识别卡后注册 sd0 块设备。DFS/elm-FatFs 使用块设备读写扇区，并按自动挂载配置将文件系统挂载到 /sd。

主参考设备：sd0。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

7.5 硬件说明

SDIO1 注册 sd0，并将 FAT/FAT32 文件系统自动挂载到 /sd。测试前插入可用 SD 卡。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

测试挂载点前先插入可用的 FAT/FAT32 SD 卡。

7.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- ../../libraries/Board_Drivers/drv_filesystem.c
- ../../libraries/gd32_drivers/drv_sdio.c
- ../../libraries/gd32_drivers/config/sdio_config.h

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_sdcard
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- ls /sd

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

7.7 运行效果

预期现象

- 插卡后 list_device 中出现 sd0。
- ls /sd 能列出 FAT/FAT32 文件和目录。
- 拔插时设备和挂载状态按配置更新，不应影响 UART1 控制台。

8. Gino_driver_sdram

8.1 简介

本工程是 GD32H77D Gino 开发板的 32 MiB 外部 SDRAM 参考工程，用于单独学习、配置和验证 SDRAM 时序、刷新与内存扩展。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

本工程没有独立的主设备节点。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查主参考设备。
free	打印 RT-Thread 堆使用情况。	确认 SDRAM 已扩展可用 heap。
sdram_test	测试 SDRAM 读写速度并校验数据，显示耗时和 MiB/s。	从 SDRAM heap 申请 1 MiB 缓冲区，读写各 8 轮，测试后释放。

8.2 SDRAM 时序、刷新与内存扩展详解

SDRAM 控制器必须按器件参数配置 bank、行列地址、CAS 延迟和刷新周期。初始化成功后，32 MiB 空间从 0xC0000000 开始并加入系统 heap；它没有独立 rt_device，因此用内存统计和压力访问验证。

一次完整的数据路径如下：

```
系统时钟 -> EXMC/SDRAM 时序与刷新 -> 0xC0000000 内存 -> RT-Thread heap
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

8.3 GD32H77D 的 EXMC 与 SDRAM 特性

GD32H77D 通过外部存储控制器配置 SDRAM 的 bank、行列地址、CAS 延迟和刷新周期。板载 32 MiB SDRAM 从 0xC0000000 开始，访问性能和稳定性受系统时钟、总线竞争与器件时序影响。

8.4 RT-Thread 内存管理接口

SDRAM 不注册独立 rt_device。BSP 初始化成功后调用内存管理接口扩展 heap，应用通过 rt_malloc、rt_calloc 和 rt_free 间接使用；free 用于查看总量和剩余空间。

本工程没有独立的主设备节点。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

8.5 硬件说明

初始化 0xC0000000 起始的 32 MiB SDRAM 并扩展系统 heap；本工程没有独立设备节点。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

后续若启用显示或摄像头功能，应避免把 framebuffer/camera 预留区纳入 heap。

8.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- ../../libraries/Board_Drivers/drv_sdram_port.c
- ../../libraries/gd32_drivers/drv_sdram.c
- ../../libraries/Board_Drivers/board.h
- board/SConscript

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_sdram
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- free
- sdram_test

sdram_test 无需参数，只访问从 SDRAM heap 申请的测试缓冲区，避开堆管理信息、已分配内存和显示/摄像头预留区；空间不足或 SDRAM 初始化失败时会报错退出。

输出表格中的 Write、Read 分别表示顺序写入、读取速度，Total KiB 为各阶段累计访问量，Time ms 为耗时，MiB/s 为每秒传输的 MiB（1 MiB = 1024 × 1024 字节）。读写各累计访问 8 MiB，数据校验单独执行并输出 Verify: PASS 或首个错误的地址、期望值和实际值。

测速使用当前 CPU、Cache 和 SDRAM 时序配置，写入计时包含每轮 D-Cache clean，读取计时包含每轮 invalidate，避免只读取写入后留在缓存中的数据。结果包含循环和系统调度开销，表示 CPU 顺序访问的有效吞吐率；不是 SDRAM 总线理论带宽。计时不足一个系统 tick 时显示 N/A。

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

8.7 运行效果

预期现象

- 启动日志显示 SDRAM 初始化成功。
- free 显示的可用 heap 明显大于仅使用内部 RAM 的工程。
- sdram_test 显示 Write、Read 的耗时和 MiB/s，并输出 Verify: PASS。
- 持续分配、写入、校验和释放大块内存时数据保持一致。

9. Gino_driver_spi

9.1 简介

本工程是 GD32H77D Gino 开发板的 SPI3 总线参考工程，用于单独学习、配置和验证 SPI 同步串行协议。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：spi3。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 spi3。
spi_loopback [length] [count]	执行 SPI3 全双工回环测试，逐字节比较收发数据。	spi_loopback 默认每轮 32 字节、测试 10 轮；spi_loopback 256 100 测试 100 轮，每轮 256 字节。
list_device	查看系统已注册设备。	确认 UART、PIN、总线和目标设备存在。

9.2 SPI 同步串行协议详解

SPI 由主机产生 SCK，并通过 MOSI/MISO 全双工移位；每个从设备还需要独立片选。当前工程支持验证 spi3 总线注册和外部回环收发。回环命令使用 8 位数据、Mode 0、MSB 优先，请求最大时钟为 1 MHz，不使用片选。

一次完整的数据路径如下：

```
应用 -> RT-Thread SPI bus -> SPI3 -> SCK/MOSI/MISO -> 外部从设备
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

9.3 GD32H77D 的 SPI3 特性

工程启用 GD32H77D SPI3，默认使用 PE12/PF0/PF1 作为时钟和数据信号。硬件控制器支持可配置的时钟分频、CPOL/CPHA 和数据宽度；片选通常由附加的 SPI device 或 GPIO 管理。

9.4 RT-Thread SPI 总线设备接口

RT-Thread 将控制器注册为 spi3 总线。具体从设备需通过 rt_spi_bus_attach_device 或 BSP 封装附加，然后使用 rt_spi_configure、rt_spi_transfer、rt_spi_send/rt_spi_recv。

主参考设备：spi3。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

9.5 硬件说明

SPI3 默认使用 PE12/PF0/PF1；具体从设备的片选、模式和时钟由应用配置。

外部回环测试前，先断开外部 SPI 从设备，再将 **PF1（MOSI）**与 **PF0（MISO）**直接短接。**PE12（SCK）**无需短接，可接逻辑分析仪观察时钟；无需连接 CS。如果修改过 SPI3 引脚映射，请按实际配置接线。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

SPI 总线注册后空闲时不会产生波形，需要具体 SPI 设备发起传输。

9.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/main.c
- applications/device_probe.c
- ../../libraries/gd32_drivers/drv_spi.c
- ../../libraries/gd32_drivers/config/spi_config.h

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_spi
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- list_device
- spi_loopback：每轮 32 字节，测试 10 轮。
- spi_loopback 256 100：每轮 256 字节，测试 100 轮。
- spi_loopback --help：查看参数范围和接线说明。

length 范围为 1-256 字节，count 范围为 1-1000 轮，均使用十进制。命令首次运行时将 spi3_loop 设备挂载到 spi3，后续运行复用该设备。每轮发送变化的数据并同步接收，检查实际传输长度，再逐字节比较。当前 BSP 使用轮询收发，实际 SCK 频率由外设时钟和分频系数决定。

运行步骤

1. 断开外部 SPI 从设备，将 PF1（MOSI）与 PF0（MISO）短接。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 执行 gino_device_probe，再运行 spi_loopback。默认测试通过时输出 SPI loopback PASS: 10 rounds, 320 bytes checked.。
5. 执行 spi_loopback 256 100 进行多轮测试。失败时根据输出的传输长度，或轮次、从 0 开始的字节偏移及 TX/RX 值，检查接线和引脚配置。

9.7 运行效果

预期现象

运行日志使用 ulog，显示时间、级别和标签：启动日志为 spi.demo，设备探测和回环测试为 spi.test。支持 ANSI 颜色的串口终端中，INFO/PASS 显示绿色，参数错误的 WARN 显示黄色，ERROR/FAIL 显示红色；命令帮助通过 ulog 原样输出。

- list_device 中出现 spi3。
- gino_device_probe 能找到 SPI3 总线。
- spi_loopback 运行时产生时钟和数据波形，收发数据全部一致时输出 PASS。
- 未接回环线或接线错误时，通常输出 FAIL 和首个不匹配字节。总线注册成功不能替代收发验证；当前驱动的轮询等待尚无超时，控制器或时钟异常时命令可能阻塞。

10. Gino_driver_usb_device

10.1 简介

本工程是 GD32H77D Gino 开发板的 USBHS0 Device 参考工程，用于单独学习、配置和验证 USB Device 枚举与 CDC 类。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：usbd。

CDC 数据设备：vcom。电脑打开 USB 虚拟串口并启用 DTR 后，应用每秒发送一条带序号的测试信息，并将收到的数据原样回显。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 usbd。
list_device	查看系统已注册设备。	确认 UART、PIN、总线和目标设备存在。

10.2 USB Device 枚举与 CDC 类详解

USBHS0 工作在 Device 角色，连接 PC 后响应总线复位和枚举请求，返回设备、配置、接口和端点描述符。当前配置启用 RT-Thread USB Device 与 CDC 组合类，PC 根据描述符加载相应驱动。

一次完整的数据路径如下：

```
PC Host -> Type-C D+/D- -> USBHS0 Device -> RT-Thread USB Device/CDC -> 应用端点
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

10.3 GD32H77D 的 USBHS0 Device 特性

工程将 GD32H77D USBHS0 配置为 Device，使用 Type-C 接口 PA11/PA12。控制器响应 Host 的复位和端点事务；当前 RT-Thread 配置启用 USB Device、Composite 和 CDC 类。

10.4 RT-Thread USB Device 设备接口

RT-Thread USB Device 栈通过描述符定义设备、配置、接口和端点，并将类请求分派给 CDC 功能。该栈不是普通字符 rt_device 的简单替代，应用通常通过 USB 类回调或类设备接口收发数据。

底层控制器注册为 usbd，CDC 串口接口注册为 vcom。应用通过 rt_device_open() 打开 vcom，使用 rt_device_read() / rt_device_write() 接收和回显数据，并在发送前通过 RT_USBD_CLASS_CTRL_CONNECTED 检查连接状态。该状态由电脑串口工具设置的 DTR 信号决定。rt_kprintf() 和 FinSH/MSH 仍使用 uart1。

10.5 硬件说明

USBHS0 Device 使用 Type-C 接口和 PA11/PA12。请使用支持数据传输的 USB 线。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

使用可传输数据的 USB 线，并确认 PC 发生枚举，而不是只给开发板供电。

10.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- ../../libraries/gd32_drivers/usb/drv_otghs_dev.c
- ../../libraries/gd32_drivers/usb/drv_usb_dev.c
- ../../libraries/gd32_drivers/usb/gd32h7_usb_hw.c
- rtconfig.h

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_usb_device
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- list_device

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 在 UART1 控制台执行 list_device，确认 usbd 和 vcom 均已注册。gino_device_probe 检查的是 usbd 控制器。
4. 将 USBHS0 Type-C 接口连接电脑，在串口工具中打开枚举出的 USB 虚拟串口，参数选 115200-8-N-1，并启用 DTR。
5. 确认每秒收到一条带序号的测试信息。发送 hello\r\n 等短字符串，确认收到原样回显；关闭电脑串口工具的本地回显，避免把本地显示误认为设备回复。
6. 关闭并重新打开 USB 虚拟串口，启用 DTR 后确认周期数据恢复。本示例用于低速通信验证，接收缓冲区为 128 字节，回显测试请使用短数据包。

10.7 运行效果

预期现象

- 连接支持数据的 USB 线后，PC 能检测到新 USB/CDC 设备。
- 设备管理器中无反复连接、断开或未知设备。
- UART1 控制台保持可用，可用于观察枚举日志。
- 打开 USB 虚拟串口并启用 DTR 后，每秒收到一条数据：

```
Gino USB CDC test: 0
Gino USB CDC test: 1
Gino USB CDC test: 2
```

- 电脑发送的数据会原样回显，周期测试信息与回显数据共用同一个串口。
- DTR 关闭或 USB 断开时暂停发送，重新打开串口并启用 DTR 后恢复。

11. Gino_driver_usb_host

11.1 简介

本工程是 GD32H77D Gino 开发板的 USBHS1 Host 与 U 盘参考工程，用于单独学习、配置和验证 USB Host 枚举与大容量存储。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：usbh。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查 usbh。
list_device	查看系统已注册设备。	确认 UART、PIN、总线和目标设备存在。
ls /udisk	列出已挂载 USB 大容量存储文件系统中的文件。	需要已枚举的 FAT/FAT32 U 盘。

11.2 USB Host 枚举与大容量存储详解

USBHS1 工作在 Host 角色，PF2 控制 VBUS 给下游设备供电。Host 栈负责检测、复位、读取描述符和选择类驱动；大容量存储类随后注册块设备，并由 DFS/FatFs 挂载为 /udisk。

一次完整的数据路径如下：

```
PF2 VBUS -> USB-A U盘 -> USBHS1 Host 枚举 -> Mass Storage -> 块设备 -> FatFs -> /
  udisk
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

11.3 GD32H77D 的 USBHS1 Host 特性

工程将 GD32H77D USBHS1 配置为 Host，USB-A 接口由 PF2 控制 VBUS。Host 控制器负责端口复位、控制传输和各类 pipe 事务，RT-Thread Host 栈完成描述符解析和类驱动匹配。

11.4 RT-Thread USB Host 与 DFS 设备接口

大容量存储类识别 U 盘后向 RT-Thread 注册块设备，DFS/FatFs 再挂载为 /udisk。应用应通过 POSIX/DFS 文件接口访问文件，而不是直接操作 USB 传输 pipe。

主参考设备：usbh。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

11.5 硬件说明

USBHS1 Host 使用 USB-A 接口，PF2 控制 VBUS；兼容 U 盘挂载到 /udisk。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

若无设备枚举，先测量 USB-A 口 VBUS。

11.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- ../../libraries/gd32_drivers/usb/drv_otghs_host.c
- ../../libraries/gd32_drivers/usb/drv_usb_host.c
- ../../libraries/gd32_drivers/usb/gd32h7_usb_hw.c
- ../../libraries/Board_Drivers/drv_filesystem.c

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_driver_usb_host
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe
- list_device
- ls /udisk

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

11.7 运行效果

预期现象

- 插入兼容 U 盘后出现 USB 枚举和 mass-storage 日志。
- list_device 能看到 Host/存储相关设备。
- ls /udisk 能列出 FAT/FAT32 文件。

显示与摄像头

1. Gino_display_camera

1.1 简介

本示例在 GD32H77D Gino 开发板上实现 OV7670 图像采集，通过 LVGL 和 MIPI LCD 实时预览。LVGL 初始化后自动启动预览。

主设备为 ov7670。UART1 用作 FinSH/MSH 控制台，参数为 115200-8-N-1，PC4 LED 用作运行指示。

1.2 采集与显示

I2C2/SCCB 配置 OV7670，TIMER0 通过 PE9 提供 XCLK。DCI 接收并行像素流，DMA1 通道 7 将数据写入内部 AXI SRAM 的独立采集缓冲。完成 Cache 维护和 RGB565 数据处理后，rt_device_read 将一帧 QVGA 图像复制到应用提供的目标缓冲。

预览线程另外使用两个内部 AXI SRAM 缓冲，跟踪正在写入、就绪和正在显示的缓冲，保证 LVGL 图像源在渲染完成前保持稳定。LVGL 通过 32 行绘制缓冲渲染，IPA 更新 SDRAM 后台帧缓冲，LCD 驱动在垂直消隐期交换帧缓冲。

1.3 内存要求

DMA 采集缓冲及两个预览缓冲均按 32 字节对齐。LCD 帧缓冲使用 SDRAM 预留区；启用 LCD 时，SDRAM heap 从 0xC0300000 开始。

板级初始化完成 SDRAM 和 MPU 配置后再开启 D-Cache。LCD 的使用依赖 SDRAM 成功初始化。通过 bootloader 跳转时，由 bootloader 在交接前清理并关闭 D-Cache。

1.4 设备接口

ov7670 设备支持 rt_device_open 和 rt_device_read。输出缓冲需要容纳一帧 320 x 240 RGB565 图像，读取成功时返回帧数据的字节数。

预览使用 lcd 图形设备和 LVGL image API，在 LVGL 线程中更新界面。启用 GT911 时提供触摸输入。

1.5 硬件说明

- OV7670：连接供电、共地、复位、XCLK、PCLK、VSYNC、HREF、D0-D7 和 I2C2/SCCB。
- 摄像头控制：I2C2 使用 PA8/PC9，XCLK 使用 PE9。
- 显示：MIPI LCD 和 SDRAM；GT911 触摸使用 I2C3。
- 控制台：UART1，PA2/PA3，AF7，115200-8-N-1。
- 运行 LED：PC4，每 500 ms 翻转。

断电后连接摄像头，供电和信号电平应满足模块要求。

1.6 示例使用

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/main.c
- applications/lv_port.c
- applications/ov7670_preview.c
- ../../libraries/Board_Drivers/drv_ov7670.c
- ../../libraries/Board_Drivers/ports/camera/sensors/ov7670.c

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_display_camera
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。本工程还会链接离线软件包 LVGL-latest、gt911-latest。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令与步骤

命令	作用
ov7670_preview start	请求在 LCD 上启动实时预览
ov7670_preview stop	停止实时预览并释放预览界面

1. 连接摄像头、LCD、电源和控制台。
2. 通过 RT-Thread Studio 或 MDK5 编译下载本工程。
3. 复位开发板，等待 LCD 自动显示摄像头画面。
4. 需要手动控制预览时，使用 start 和 stop 命令。

1.7 预期效果

- LVGL 初始化显示设备并启动 OV7670 预览。
- LCD 持续显示 QVGA RGB565 摄像头图像。
- 可通过控制台停止并重新启动预览。

2. Gino_display_gd32embeddedui

GD32H77D Gino 开发板运行 GD32EmbeddedUI 导出界面的独立工程。使用 **LVGL 8.3.11**、**720×720 MIPI DSI LCD**、**RGB565**、**GT911 触摸**。

2.1 可编辑的 UI 软件包

UI 放在共享软件包*../../packages/GD32EmbeddedUI-demos*，通过本工程 packages/GD32EmbeddedUI-demos 目录链接引用。直接用 GD32EmbeddedUI 打开软件包内的*GD32Embeddedui.gdlvgl*，即可继续编辑、保存和生成代码，板端构建会读取同一份导出结果。

当前附带空白设计工程，custom_init() 保留为应用初始化入口。可以在设计器中添加标签或按钮，保存并生成代码后验证显示和触摸。

2.2 硬件与运行流程

复用 Gino_display_lvgl8 的板级配置和显示/触摸适配：

- LCD、GT911、SDRAM 由现有板级驱动初始化。
- 两个 200 行 RGB565 绘制缓冲位于 AXI SRAM，IPA 将绘制结果复制到 SDRAM 帧缓冲。
- RT-Thread 的 LVGL 线程依次初始化 LVGL、显示、触摸，然后调用 ui_init()。
- ui_init() 内部调用 custom_init()；LVGL 线程持续处理刷新和触摸事件。

不要在 main() 中再次初始化 LVGL 或调用 ui_init()。应用逻辑写在软件包的 custom/ 中；从其他线程更新 UI 时，需要与 LVGL 线程进行同步。

2.3 打开板级工程

在本工程目录执行：

```
.\mklinks.bat
scons --target=mdk5
```

然后用 MDK5 打开 project.uvprojx。scons --target=mdk5 只生成工程文件。使用 RT-Thread Studio 时，可导入当前目录，或从 SDK 示例列表创建本工程。Studio 更新文件列表使用：

```
scons --target=eclipse
```

执行后刷新 Studio 工程。Linux 使用 sh mklinks.sh 建立共享源码链接。本工程链接 rt-thread、libraries、LVGL-v8.3.11、gt911-latest 和 GD32EmbeddedUI-demos。SDK 示例导出为独立工程时，这些依赖会复制到工程内；此时应打开独立工程 packages/GD32EmbeddedUI-demos/GD32Embeddedui.gdlvgl。

2.4 后续修改界面

1. 打开上述 .gdlvgl 文件，在 GD32EmbeddedUI 中编辑并生成代码。
2. 保持 LVGL 8.3.11、720×720、16 位色深。
3. 图片和字体使用 C 数组/静态字体输出，放在 generated/ 下。
4. 新增或删除源文件后重新生成 MDK/Studio 工程文件，再进行固件构建和下载。

SCons 自动递归收集 generated/、custom/ 中的 .c 文件及头文件目录。applications/lv_conf.h 加载导出的 ui_config.h 和 lv_conf_patch.h。PC 模拟器、Windows 入口及第三方库不会加入板端构建。

本示例没有配置资源文件系统、FreeType 或 FFmpeg。设计器预览中使用的文件字体、图片路径和视频需要额外移植，不能仅靠复制 assets/ 在板端使用。

2.5 上板检查

- UART1 控制台输出工程名称，PC4 LED 闪烁。
- list_device 和 gino_device_probe 可以检查 lcd、gt911 等设备。
- 显示初始化成功后输出 GD32EmbeddedUI initialized (720x720, RGB565)。
- 空白设计默认没有控件；在设计器中添加按钮并生成代码后验证触摸响应。

验证范围为工程生成和相关源码语法检查；完整固件构建、下载和硬件运行仍需验证。

3. Gino_display_lvgl

3.1 简介

本工程使用 ../../packages/LVGL-latest 中的 **LVGL 9.x**（当前随 SDK 附带的是 9.6 开发版）。固定 8.3.11 版本请使用独立的 Gino_display_lvgl8 工程。

本工程是 GD32H77D Gino 开发板的 MIPI DSI LCD、GT911 与 LVGL 图形参考工程，用于单独学习、配置和验证 LVGL 渲染、输入与双缓冲。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：lcd。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查lcd。

3.2 LVGL 渲染、输入与双缓冲详解

LVGL 独立线程初始化对象系统、显示和触摸，并周期调用 lv_timer_handler 推进动画、输入和刷新。本工程使用两个位于内部 AXI SRAM 的 200 行局部绘制缓冲，由 IPA 异步复制到 SDRAM 中的单个扫描帧缓冲，传输完成后才通知 LVGL 复用绘制缓冲。

一次完整的数据路径如下：

```
GT911 输入 -> LVGL 对象/事件 -> RGB565 缓冲 -> Cache flush/IPA -> TLI/DSI -> LCD
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

3.3 GD32H77D 的 SDRAM、TLI/DSI 与 I2C3 特性

本工程组合 GD32H77D SDRAM、TLI、DSI、IPA、I2C3 和定时器 PWM。TLI/DSI 持续扫描帧缓冲，GT911 提供触摸，IPA 可加速局部复制，Cortex-M7 D-Cache 需要在显示 DMA/TLI 读取前清理。

板级初始化使用 MPU region 0 保护未使用的外部地址窗口，由更高优先级的区域放行有效 SDRAM。若使用 OSPI 内存映射，需要先为其配置独立的可访问 MPU 区域。

D-Cache 在 SDRAM 初始化和显示内存 MPU 配置完成后才开启，LCD 使用板级初始化配置的 MPU 和 Cache 属性。若由 bootloader 跳转启动，需要由 bootloader 在交接前清理并关闭 D-Cache。

3.4 RT-Thread LVGL、Graphic 与 Touch 设备接口

LVGL RT-Thread port 创建独立 LVGL 线程，调用 lv_timer_handler。显示端使用 lcd 图形设备，输入端使用 gt911 touch device。LVGL 9 使用 lv_display_set_buffers 注册缓冲区，LVGL 8 使用 lv_disp_draw_buf_init 和 lv_disp_drv_register；flush callback 在硬件使用完成后释放绘制缓冲区。

主参考设备：lcd。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

LVGL 8 与 9 兼容

Gino_display_lvgl、Gino_display_lvgl8、Gino_display_camera 和 Gino_factory 通过 LVGL_VERSION_MAJOR 自动选择接口，共用 ../../libraries/Board_Drivers/lvgl_compat.h。适配覆盖显示刷新和触摸回调类型、图片及控件接口名称，各工程的 lv_conf.h 配置原生 RGB565。

在 SDK 仓库内使用时，先运行一次 mklinks.bat（Linux 下运行 sh mklinks.sh），建立 RT-Thread、共享板级库、GT911 和 LVGL-latest 的工程链接。通过 Studio 的 SDK 示例入口创建独立工程时，清单会复制对应的离线软件包。packages/SConscript 只选择 .config 和 rtconfig.h 配置的 LVGL 版本，排除目录中残留的其他版本。主动更改软件包版本后，需重新生成 MDK/Studio 工程配置。LVGL_VERSION_MAJOR 由 LVGL 自身提供，不要手动修改。

相关源文件已通过 LVGL 8.3.11、8.4.0 和仓库内 9.6 开发版的语法检查。8.x 缓冲区大小按像素数填写，9.x 按字节数填写。摄像头和出厂工程关闭摄像头支持、使用两个扫描帧缓冲时，8.x 在垂直消隐换帧前绘制完整画面，9.x 使用 direct 渲染。刷新率和触摸效果仍需上板验证。

3.5 硬件说明

整合 LCD、GT911、SDRAM 和 LVGL 离线包。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

操作屏幕，确认动画和触摸响应持续正常。

3.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/lv_port.c
- applications/lv_conf.h
- ../../libraries/Board_Drivers/drv_lcd.c
- ../../libraries/Board_Drivers/lvgl_compat.h
- ../../packages/LVGL-latest/env_support/rt-thread/lv_rt_thread_port.c

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_display_lvgl
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。本工程还会链接离线软件包 LVGL-latest、gt911-latest。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

3.7 运行效果

预期现象

- 启动后显示 LVGL demo，触摸可驱动界面控件。
- 绘制缓冲交替使用，IPA 持续将界面更新到 LCD。

4. Gino_display_lvgl8

4.1 简介

本工程固定使用 **LVGL 8.3.11**，离线源码位于 ../../packages/LVGL-v8.3.11。9.x 版本请使用独立的 Gino_display_lvgl 工程。

本工程是 GD32H77D Gino 开发板的 MIPI DSI LCD、GT911 与 LVGL 图形参考工程，用于单独学习、配置和验证 LVGL 渲染、输入与双缓冲。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：lcd。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查lcd。

4.2 LVGL 渲染、输入与双缓冲详解

LVGL 独立线程初始化对象系统、显示和触摸，并周期调用 lv_timer_handler 推进动画、输入和刷新。本工程使用两个位于内部 AXI SRAM 的 200 行局部绘制缓冲，由 IPA 异步复制到 SDRAM 中的单个扫描帧缓冲，传输完成后才通知 LVGL 复用绘制缓冲。

一次完整的数据路径如下：

```
GT911 输入 -> LVGL 对象/事件 -> RGB565 缓冲 -> Cache flush/IPA -> TLI/DSI -> LCD
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

4.3 GD32H77D 的 SDRAM、TLI/DSI 与 I2C3 特性

本工程组合 GD32H77D SDRAM、TLI、DSI、IPA、I2C3 和定时器 PWM。TLI/DSI 持续扫描帧缓冲，GT911 提供触摸，IPA 可加速局部复制，Cortex-M7 D-Cache 需要在显示 DMA/TLI 读取前清理。

板级初始化使用 MPU region 0 保护未使用的外部地址窗口，由更高优先级的区域放行有效 SDRAM。若使用 OSPI 内存映射，需要先为其配置独立的可访问 MPU 区域。

D-Cache 在 SDRAM 初始化和显示内存 MPU 配置完成后才开启，LCD 使用板级初始化配置的 MPU 和 Cache 属性。若由 bootloader 跳转启动，需要由 bootloader 在交接前清理并关闭 D-Cache。

4.4 RT-Thread LVGL、Graphic 与 Touch 设备接口

LVGL RT-Thread port 创建独立 LVGL 线程，调用 lv_timer_handler。显示端使用 lcd 图形设备，输入端使用 gt911 touch device。LVGL 9 使用 lv_display_set_buffers 注册缓冲区，LVGL 8 使用 lv_disp_draw_buf_init 和 lv_disp_drv_register；flush callback 在硬件使用完成后释放绘制缓冲区。

主参考设备：lcd。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

LVGL 8 与 9 兼容

Gino_display_lvgl、Gino_display_lvgl8、Gino_display_camera 和 Gino_factory 通过 LVGL_VERSION_MAJOR 自动选择接口，共用 ../../libraries/Board_Drivers/lvgl_compat.h。适配覆盖显示刷新和触摸回调类型、图片及控件接口名称，各工程的 lv_conf.h 配置原生 RGB565。

在 SDK 仓库内使用时，先运行一次 mklinks.bat（Linux 下运行 sh mklinks.sh），建立 RT-Thread、共享板级库、GT911 和 LVGL-v8.3.11 的工程链接。通过 Studio 的 SDK 示例入口创建独立工程时，清单会复制对应的离线软件包。packages/SConscript 只选择 .config 和 rtconfig.h 配置的 LVGL 版本，排除目录中残留的其他版本。本工程同时在编译时检查源码版本，要求使用 8.3.11。使用 9.x 时请选择 Gino_display_lvgl。LVGL_VERSION_MAJOR 由 LVGL 自身提供，不要手动修改。

相关源文件已通过 LVGL 8.3.11、8.4.0 和仓库内 9.6 开发版的语法检查。8.x 缓冲区大小按像素数填写，9.x 按字节数填写。摄像头和出厂工程关闭摄像头支持、使用两个扫描帧缓冲时，8.x 在垂直消隐换帧前绘制完整画面，9.x 使用 direct 渲染。刷新率和触摸效果仍需上板验证。

4.5 硬件说明

整合 LCD、GT911、SDRAM 和 LVGL 离线包。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

操作屏幕，确认动画和触摸响应持续正常。

4.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/lv_port.c
- applications/lv_conf.h
- ../../libraries/Board_Drivers/drv_lcd.c
- ../../libraries/Board_Drivers/lvgl_compat.h
- ../../packages/LVGL-v8.3.11/env_support/rt-thread/lv_rt_thread_port.c

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_display_lvgl8
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。本工程还会链接离线软件包 LVGL-v8.3.11、gt911-latest。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- gino_device_probe

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

4.7 运行效果

预期现象

- 启动后显示 LVGL demo，触摸可驱动界面控件。
- 绘制缓冲交替使用，IPA 持续将界面更新到 LCD。

组件示例

1. Gino_component_mqtt

1.1 简介

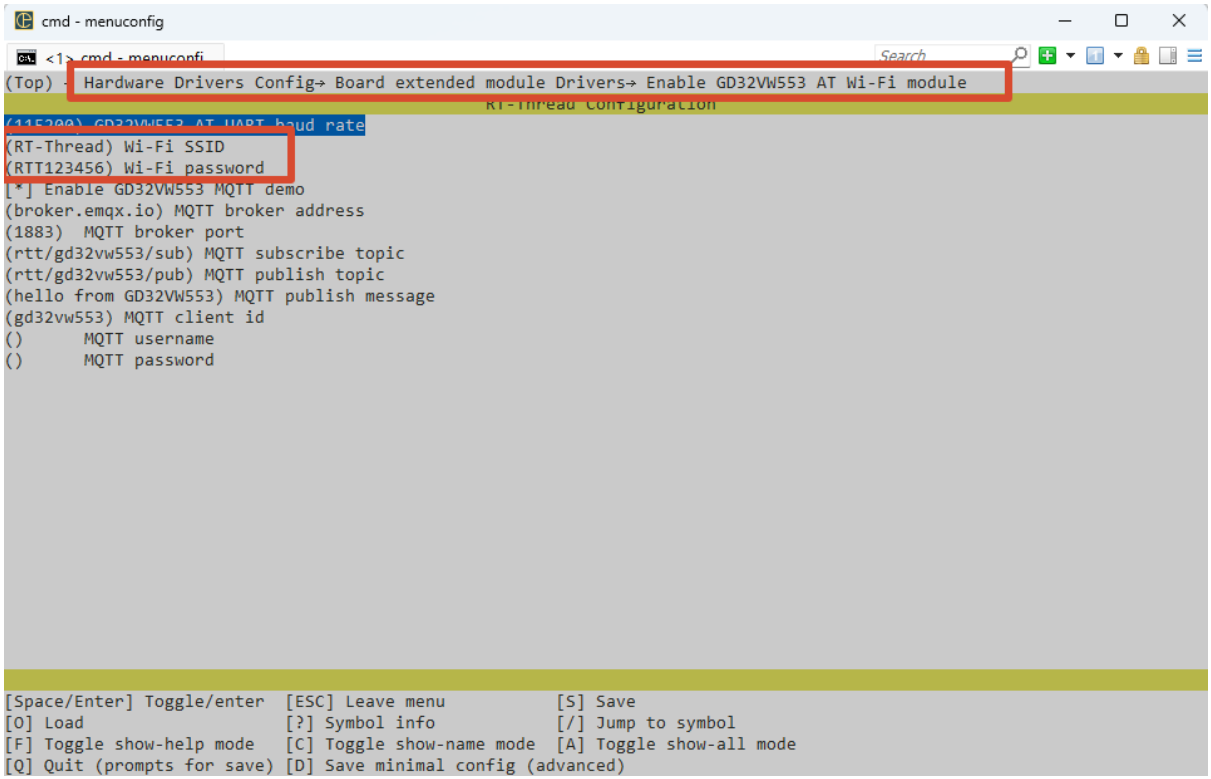
本工程是 GD32H77D Gino 开发板的 GD32VW553、SAL 与 kawaii-mqtt 发布订阅参考工程，用于单独学习、配置和验证 MQTT 发布订阅协议。独立工程只启用该功能需要的驱动和组件，可作为应用开发和功能集成的参考。

主参考设备：wifi0。

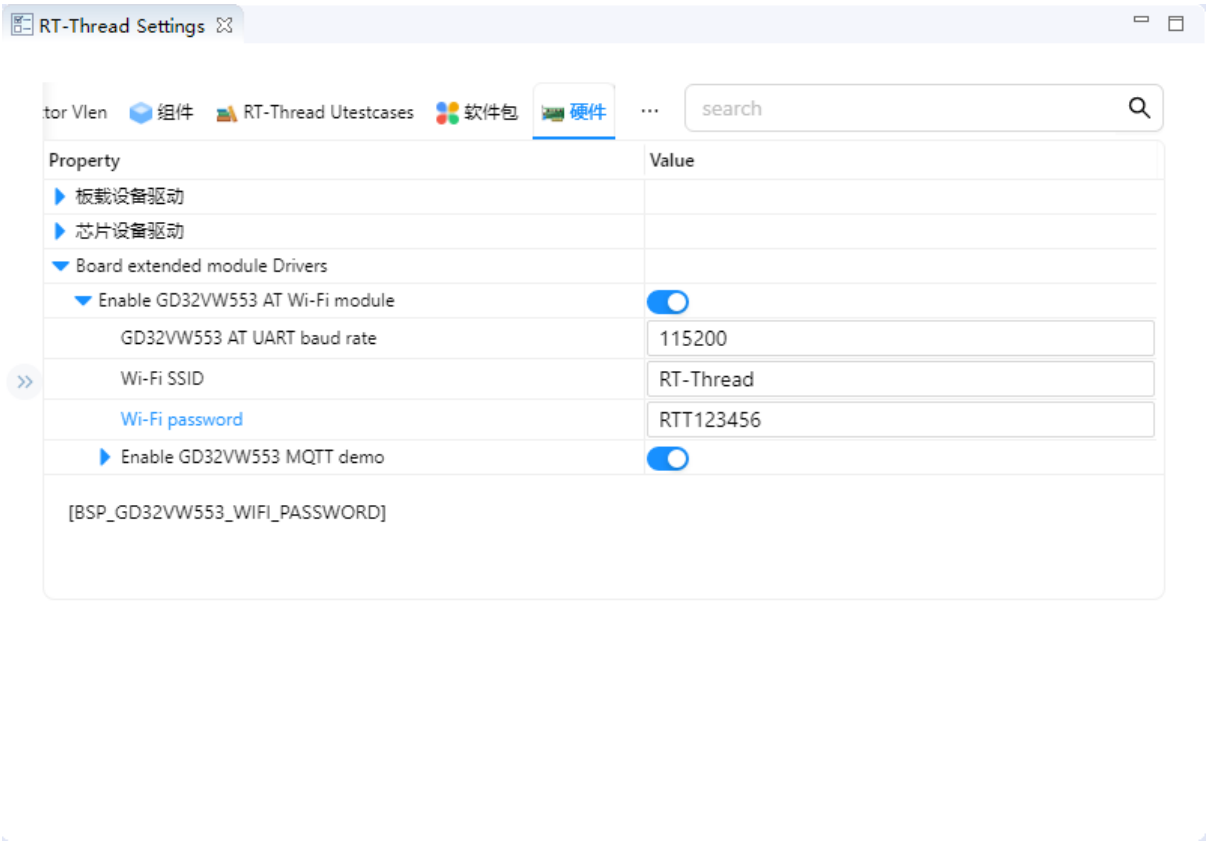
所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

运行示例的时候需要修改成自己的 wifi

env 中的修改方式



studio 中的修改方式



当前工程包含以下 MSH 命令：

命令	作用	示例/备注
gd32vw553_mqtt_start	启动 GD32VW553 MQTT 示例，等待wifi0 联网后连接 Broker 并订阅主题。	gd32vw553_mqtt_start
gd32vw553_mqtt_pub <topic> <message>	通过当前 MQTT 连接发布一条消息。	gd32vw553_mqtt_pub rtt/gino hello
gd32vw553_mqtt_stop	停止 MQTT 示例，断开 MQTT/TCP 连接并释放客户端资源。	gd32vw553_mqtt_stop
ifconfig	查看wifi0 的联网状态和 IP 地址。	确认网络已 ready。
list_device	查看系统已注册设备。	确认 UART、PIN、netdev 等依赖设备存在。
gino_device_probe	检查当前示例使用的主要设备是否已注册。	默认检查wifi0。

注意使用示例的时候可以使用自己所定义的主题。

1.2 MQTT 发布订阅协议详解

示例先等待 wifi0 ready，并临时将其设为 SAL 默认网卡。kawaii-mqtt 负责 MQTT 编解码和会话，底层 socket 经 SAL、AT socket 和 UART4 到 GD32VW553。发布命令直接调用当前 MQTT client，并把真实发送结果返回给 MSH。

一次完整的数据路径如下：

```
MSH publish -> kawaii-mqtt -> socket/SAL -> AT socket -> UART4 -> GD32VW553 ->
  Broker
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据收发验证。

1.3 GD32H77D 的 UART4 与 Cortex-M7 特性

GD32H77D 侧主要使用 Cortex-M7、UART4、RT-Thread IPC 和内存资源；实际 Wi-Fi/TCP/IP 由 GD32VW553 卸载。MQTT 报文仍在 H7 上由 kawaii-mqtt 编解码，并经 SAL/AT socket 发送。

1.4 RT-Thread AT Device、SAL 与 MQTT 设备接口

网络基础是 wifi0 at_device/netdev 与 SAL socket。示例用 RT-Thread mutex 保护客户端状态，并调用 mqtt_connect、mqtt_subscribe 和 mqtt_publish。

主参考设备：wifi0。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

1.5 硬件说明

GD32VW553 使用 UART4（PB12/PB13）通信，PC5 控制复位。需在本地配置 Wi-Fi、Broker、主题和认证信息。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

1.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/gd32vw553_mqtt_demo.c
- ../../packages/at_device-latest/samples/at_sample_gd32vw553.c
- ../../packages/kawaii-mqtt-latest/
- ../../packages/at_device-latest/class/gd32vw553/

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_component_mqtt
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。本工程还会链接离线软件包 at_device-latest、kawaii-mqtt-latest。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- ifconfig
- gd32vw553_mqtt_start
- gd32vw553_mqtt_pub rtt/gino hello
- gd32vw553_mqtt_stop

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

1.7 运行效果

预期现象

- ifconfig 显示 wifi0 已联网。
- 启动命令完成 TCP/MQTT 连接并订阅目标主题。
- 发布后 Broker/订阅端收到 payload，接收回调能打印订阅消息；停止命令释放客户端并恢复默认 netdev。

出厂演示

1. Gino_factory

1.1 简介

本工程为 GD32H77D Gino 开发板提供 LVGL、摄像头、MQTT、以太网、存储与常用外设综合演示，通过触摸界面和控制台使用各项功能。

主参考设备：lcd。

所有示例均保留 uart1 作为 FinSH/MSH 控制台，串口参数为 115200-8-N-1；PC4 LED 用于运行指示。

当前工程包含以下 MSH 命令：

命令	作用	示例/备注
ov7670_preview start	请求在 LCD 上启动摄像头实时预览。	显示和摄像头 ready 后使用。
gd32vw553_mqtt_start	启动 GD32VW553 MQTT 示例。	连接 Wi-Fi、Broker 和订阅路径。
ifconfig	查看网络设备的联网状态和 IP 地址。	确认网络已 ready。
list_device	查看系统已注册设备。	确认 UART、PIN、总线和目标设备存在。
i2c scan hwi2c1 08 78	扫描外部 I2C 地址 0x08-0x77。	由 i2c-tools 软件包提供，外部总线为 hwi2c1。

触摸界面的 I2C 页面固定扫描外部 hwi2c1（SCL PH4、SDA PB11），点击 Scan 后由 board_demo_backend.c 使用零数据长度的传输扫描地址。hwi2c2 和 hwi2c3 分别供摄像头和 GT911 触摸使用，不提供界面扫描入口。串口命令使用仓库内的 i2c-tools v1.0.0 软件包，采用硬件总线模式。打开工程前，Windows 执行 mklinks.bat，Linux 执行 sh mklinks.sh。执行 i2c 可查看扫描、读写命令用法；扫描范围按十六进制解析，结束地址不包含在内。

SPI 页面通过 SPI3 执行真实的全双工回环测试：短接 PF1（MOSI）与 PF0（MISO），SCK 为 PE12，使用 Mode 0、8 位数据且不使用片选。选择最大时钟频率和传输长度后点击 Run loopback，屏幕日志显示排队、配置、完整 TX/RX 十六进制数据及 PASS/FAIL；数据不一致时显示首个错误字节的位置和收发值。

I2C 页面使用与 i2c-tools 相同的 16 列地址矩阵，响应设备直接显示地址，无响应显示 --，未扫描显示 ..，探测错误显示 !!，保留地址留空。扫描范围为 0x08-0x77，结果逐地址更新，完成后显示设备总数和耗时。扫描不再逐地址休眠；获取总线锁最多等待 100 ms，持锁期间临时

将驱动等待阶段的超时限制到 20 ms，探测结束后恢复原值。遇到超时或其他总线错误立即停止，保留已扫描结果，不把未完成的扫描误报为“无设备”。

Scan log 保留响应地址、异常和最终结果。SPI、I2C 测试执行期间禁用各自按钮和参数选择，提交失败会显示错误。日志同步输出到串口，每项保留本次任务最近 3 KB，屏幕可滚动查看。这些屏幕日志对应触摸界面发起的任务；I2C 页面目前只扫描地址，不提供特定芯片的数据或固件烧录功能。

Wi-Fi 配置

运行出厂示例前，需要将 Wi-Fi 名称和密码改为自己的网络信息。配置方法和截图参考 Gino_component_mqtt，但各工程的配置相互独立，需要在 Gino_factory 中修改。

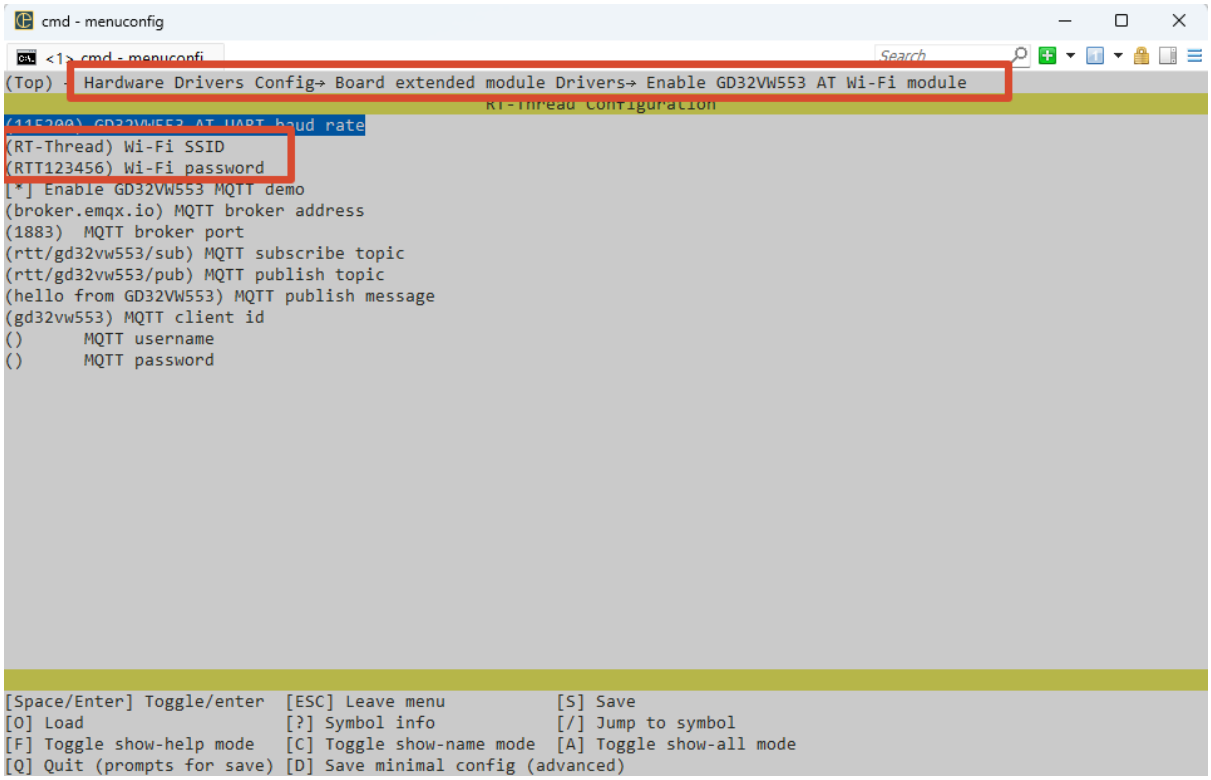
板载 GD32VW553 AT 模块使用 UART4（TX PB12、RX PB13，AF14），PC5 控制复位，网络设备名为 wifi0；UART1 仍用于控制台。

Env

1. 在 Env 中从 SDK 根目录进入 projects\Gino_factory；尚未建立共享目录链接时，先执行 mklinks.bat。
2. 执行 menuconfig，进入以下菜单：

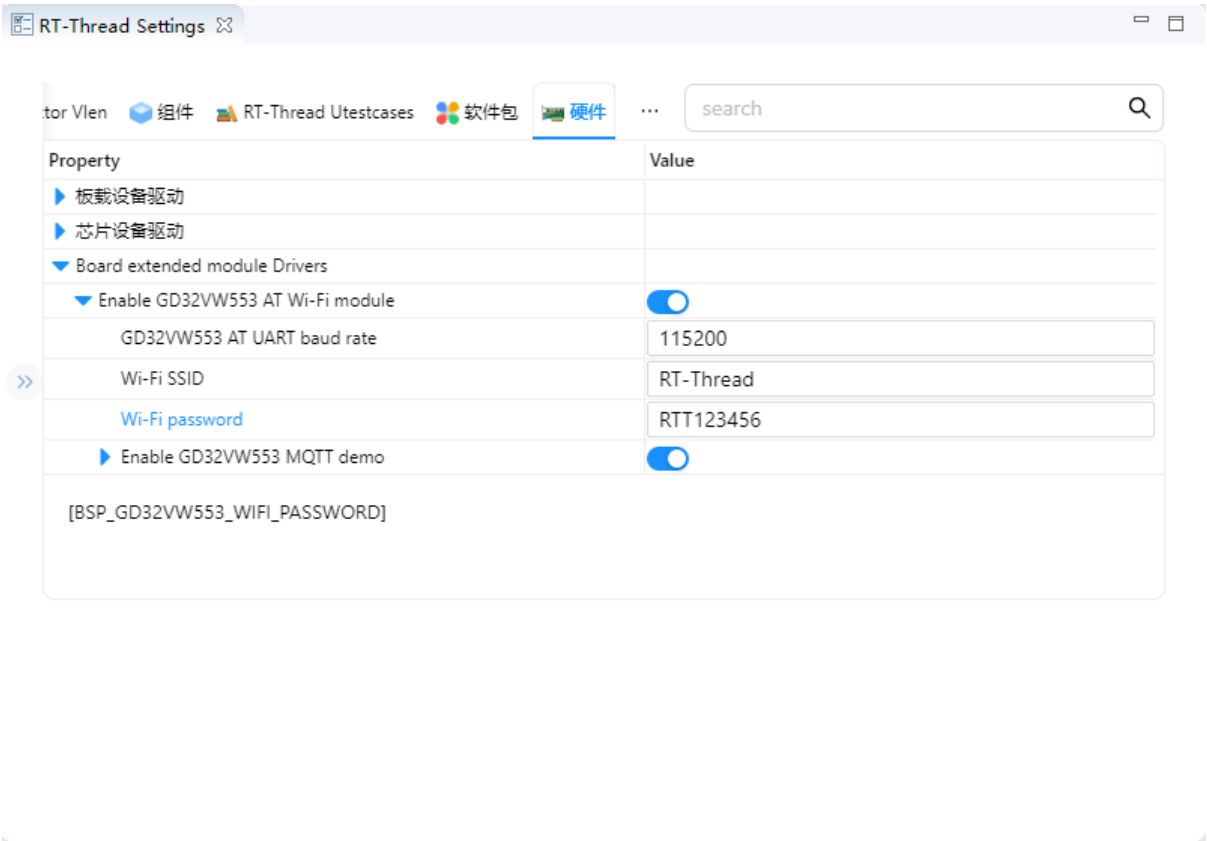
```
Hardware Drivers Config
    -> Board extended module Drivers
        -> Enable GD32VW553 AT Wi-Fi module
```

3. 保持模块启用，进入 RT-Thread online packages -> IoT - internet of things -> AT DEVICE，版本选择 latest，在 GigaDevice GD32VW553 -> Enable sample 中设置 WIFI ssid 和 WIFI password。AT UART baud rate 保持为 115200；若模块固件使用其他波特率，则与固件保持一致。板级开关会自动启用注册 sample，并将复位脚默认为 PC5（编号 37）、低电平有效。
4. 保存退出后，执行 scons --pyconfig-silent 更新 rtconfig.h。使用 MDK5 时，再执行 scons --target=mdk5 更新工程文件。



RT-Thread Studio

1. 打开出厂工程的 RT-Thread Settings，选择“硬件”页签。
2. 展开 Board extended module Drivers -> Enable GD32VW553 AT Wi-Fi module，保持模块启用。
3. 切换到软件包设置，在 AT DEVICE（latest）的 GigaDevice GD32VW553 -> Enable sample 中修改 WIFI ssid 和 WIFI password，确认 AT 串口波特率后保存。截图为此前的配置布局，Wi-Fi 参数现已统一到软件包 sample 菜单。



这两个配置项分别对应 GD32VW553_SAMPLE_WIFI_SSID 和 GD32VW553_SAMPLE_WIFI_PASSWORD。请填写自己的网络信息，截图中的数值仅作示例。保存后重新编译、下载出厂固件，并复位开发板，使配置生效。

Wi-Fi 与 MQTT 验证

在 UART1 控制台执行 ifconfig，确认 wifi0 已启用、链路已连接，且 IP 地址不是 0.0.0.0。出厂工程同时启用了以太网 e0，以太网已连接不代表 Wi-Fi 已就绪。若 wifi0 未联网，检查 SSID、密码、热点是否可用、模块供电与复位，以及 AT 串口波特率。

使用 MQTT 时，在板级 GD32VW553 配置菜单中保持 Enable GD32VW553 MQTT demo 启用，并按自己的服务设置 Broker 地址/端口、订阅/发布主题、client ID 和可选的用户名/密码。主题可以自行定义。模块固件不支持 AT+CIPDOMAIN DNS 查询时，使用数字 IPv4 Broker 地址。

配置生效并确认 wifi0 就绪后，执行：

```
gd32vw553_mqtt_start
gd32vw553_mqtt_pub rtt/gino hello
gd32vw553_mqtt_stop
```

等待 MQTT 连接和订阅成功后再发布，将 rtt/gino 替换为自己的主题，并在 Broker/订阅端确认收到消息。MQTT 示例运行时临时选择 wifi0 为默认网络设备，停止后恢复原来的默认设备。

1.2 多外设并发与系统集成详解

综合工程用于验证各独立功能能否同时共享引脚、初始化级别、DMA、Cache、SDRAM、网络默认路由和系统 heap。显示持续从 SDRAM 扫描，摄像头和 ENET 同时使用 DMA，Wi-Fi/MQTT 通过 SAL 选择 wifi0，文件系统在设备注册后挂载；它更接近真实产品负载。

一次完整的数据路径如下：

```
各驱动注册 -> DFS/lwIP/LVGL/SAL -> 显示 + 摄像头 + 存储 + e0/wifi0 + MQTT
  并发运行
```

协议层只定义通信或数据处理规则，最终仍需要控制器时钟、引脚复用、中断/DMA 和上层状态机共同工作。扫描到设备、注册了总线或编译通过，都不能替代完整的数据传输验证。

1.3 GD32H77D 的多总线、多 DMA 与存储器特性

综合工程同时使用 GD32H77D 的 SDRAM/TLI/DSI、DCI/DMA、ENET1、OSPI0、SDIO1、USBHS、CAN、SPI、I2C、UART 和定时器。重点是验证引脚、DMA、Cache、总线带宽、内存区域和中断优先级能否共存。

1.4 RT-Thread 综合设备与组件接口

RT-Thread 层同时运行设备框架、DFS/FAL/FatFs、lwIP/netdev/SAL、at_device、LVGL、touch、camera 和 MQTT 线程。应用通过统一设备名、自动初始化级别和 IPC 协调模块。

主参考设备：lcd。设备是否已注册可先通过 list_device 和 gino_device_probe 判断；更上层的文件系统、网络或 GUI 组件还需继续验证其挂载、链路或刷新状态。

1.5 硬件说明

综合启用 720 x 720 LCD、GT911、OV7670、GD32VW553、ENET1、OSPI Flash、SDIO、USB、CAN、SPI、I2C 和 PWM。各接口接线与电气条件分别遵循对应独立工程。

默认控制台：UART1，PA2/PA3，AF7，115200-8-N-1。

1.6 工程示例说明

以下源码路径以 SDK 仓库中的工程目录为基准：

- applications/board_demo.c
- applications/board_demo_backend.c
- applications/lv_port.c
- applications/ov7670_preview.c
- applications/gd32vw553_mqtt_demo.c
- board/SConscript

建议先阅读 applications/main.c 和 applications/device_probe.c，再沿数据路径进入对应驱动、组件或软件包。示例保留 MSH 命令，便于在不改动应用代码的情况下观察设备注册和运行状态。

tests/run_host_tests.ps1 使用本机 GCC 和模拟总线验证命令分发、扫描结果、超时停止及超时参数恢复，不编译固件。默认使用 MSYS2 UCRT64 GCC，可通过 -Compiler 指定路径。测试直接提取当前后端函数；不能替代上板检查接线、ACK 波形与实际扫描耗时。

MDK5 工程生成

在 Env 2.0 或更新版本的 PowerShell 环境中，从 SDK 根目录执行：

```
cd projects\Gino_factory
.\mklinks.bat
scons --target=mdk5
```

mklinks.bat 建立共享的 rt-thread 和 libraries 目录链接。本工程还会链接离线软件包 LVGL-latest、gt911-latest、at_device-latest、kawaii-mqtt-latest、i2c-tools-v1.0.0。

执行后在本目录生成 project.uvprojx。修改功能配置或源码选择后，重新执行 scons --target=mdk5 更新工程。

运行命令

- ov7670_preview start
- gd32vw553_mqtt_start
- ifconfig
- list_device
- i2c scan hwi2c1 08 78

运行步骤

1. 检查供电、接线、外部模块和接口电平。
2. 复位开发板，确认 UART1 控制台可用且 PC4 LED 正常闪烁。
3. 执行 list_device，确认依赖总线和目标设备均已注册。
4. 按顺序执行上述命令，并同时观察返回值、外部波形、网络状态或显示结果。

1.7 运行效果

预期现象

- list_device 中出现已启用的 LCD、触摸、摄像头、存储、网络和总线设备。
- LVGL、摄像头预览、以太网/Wi-Fi 和文件系统可以分别启动，并能在合理负载下共存。
- 触摸界面提供板级功能入口，控制台可操作摄像头、网络和 MQTT。
- I2C 触摸页面和 i2c scan 命令均可列出所选总线上有响应的从设备地址。

- SPI 回环页面显示收发数据及校验结果，I2C 页面显示扫描过程日志和响应地址。